Elements of AIML
Unit 01 – Lecture 02 Notes
Intelligent Agents & Learning Agents

Tofik Ali

March 28, 2026

Contents

1 Lecture Overview **2**

2 How to use these notes (self-study) **2**

3 Learning Outcomes **2**

4 Bridge from Lecture 01 **2**

5 Keywords **2**

6 Abbreviations and notation **3**

7 Agent and environment **3**
7.1 Definition . . . 3
7.2 Agent function vs agent program . . . 3
7.3 Why agents are useful . . . 3
7.4 Performance measure vs reward vs utility (intuition) . . . 3

8 PEAS task specification **4**
8.1 Example 1: spam filter . . . 4
8.2 Example 2: vacuum cleaner . . . 4
8.3 Worked design example: campus navigation assistant . . . 4

9 Environment properties **5**
9.1 Examples for environment properties . . . 5
9.2 Why classification matters . . . 5

10 Rationality and bounded rationality **5**
10.1 Rationality . . . 5
10.2 Bounded rationality . . . 5

11 Agent types (high level) **5**
11.1 Simple reflex vs model-based reflex (expanded) . . . 6
11.2 Goal-based and utility-based agents (expanded) . . . 6

12 Learning agent architecture **6**
12.1 Exploration vs exploitation (important intuition) . . . 6

13 Common pitfalls (read before exams) **6**

14 Practice (with answers) **7**

15 Exit questions (with answers) **7**

16 Summary **7**

1 Lecture Overview

This lecture introduces **intelligent agents**, the central abstraction used in many AI systems. We focus on how to specify an AI task (PEAS), how to describe environments, and how learning agents improve over time.

Sources: Rich and Knight, *Artificial Intelligence*, McGraw Hill, 2017.;Russell and Norvig, *Artificial Intelligence: A Modern Approach* (4e), Ch. 1–2 (Introduction, Intelligent Agents).;Poole and Mackworth, *Artificial Intelligence: Foundations of Computational Agents* (2e), selected sections.

2 How to use these notes (self-study)

Start by understanding the **agent view**: an AI system is something that *perceives* and *acts*. Then practice writing PEAS and classifying environments for 2–3 familiar systems (navigation app, spam filter, delivery app). These two skills make the rest of AI topics easier, because they force you to state assumptions clearly.

3 Learning Outcomes

- Define an agent and an environment in precise terms.
- Construct PEAS descriptions for real tasks.
- Classify environments and explain why classification matters.
- Compare agent types and describe a learning agent architecture.

4 Bridge from Lecture 01

Before continuing, recall three ideas from **Unit 01 Lecture 01**:

- AI systems are goal-directed, so we need a clear **performance measure**.
- The same domain can be framed as different AI tasks depending on the required **action**.
- Rational behavior means choosing actions that best support the goal under available information.

This lecture adds the missing design lens: *what exactly can the system observe, what can it do, and what type of environment is it operating in?*

5 Keywords

- **Agent:** entity that perceives (sensors) and acts (actuators).
- **Percept:** the agent's input at a time step (observation).

- **Action:** an output decision that changes the environment.
- **PEAS:** Performance measure, Environment, Actuators, Sensors.
- **Rational agent:** chooses actions that maximize expected performance.
- **Bounded rationality:** rationality constrained by time/compute/information.

6 Abbreviations and notation

- **PEAS**: Performance measure, Environment, Actuators, Sensors.
- **Percept**: the observation received from sensors.
- **Percept sequence**: the history of percepts observed so far.
- **Action**: the operation selected by the agent (through actuators).

7 Agent and environment

7.1 Definition

Agent: an agent maps percept sequences to actions:

$$f : (percepts)^* \to actions$$

This does not mean the agent stores the entire history; it means the *behavior* can depend on what has been observed so far.

7.2 Agent function vs agent program

It is useful to distinguish:

- **Agent function:** the ideal mapping from percept history to action (what we want).
- **Agent program:** the actual implementation running on hardware (what we build).

In the real world, the agent program has constraints: limited sensors, noisy data, limited compute, and deadlines. So we often aim for the **best feasible** behavior rather than perfect behavior.

7.3 Why agents are useful

The agent framework forces us to be explicit about:

- what the system can observe,
- what it can do,
- how success is measured.

These choices strongly affect what algorithms are possible.

7.4 Performance measure vs reward vs utility (intuition)

In many AI discussions, three related terms appear:

- **Performance measure:** external definition of success (what humans truly care about).

- **Reward:** feedback signal given during interaction (common in reinforcement learning).
- **Utility:** internal score used by an agent to compare actions (trade-offs).

Good system design aligns reward/utility with the real performance measure; misalignment can lead to unintended or unsafe behaviors.

8 PEAS task specification

PEAS provides a structured way to describe an agent task.

8.1 Example 1: spam filter

- **Performance:** reduce spam in inbox; avoid blocking important mail.
- **Environment:** email stream + user preferences + mail server policies.
- **Actuators:** classify/move to spam, block sender, flag, allow.
- **Sensors:** email content, metadata, links, user feedback.

8.2 Example 2: vacuum cleaner

- **Performance:** cleanliness, time, energy, collision avoidance.
- **Environment:** rooms, obstacles, people/pets, dirt distribution.
- **Actuators:** move, turn, suck, dock/charge.
- **Sensors:** camera/LiDAR, bump sensor, dirt sensor, battery level.

8.3 Worked design example: campus navigation assistant

Suppose the problem statement is: "Build an assistant that recommends safe and fast walking routes inside a university campus." A complete design flow looks like this:

1. **PEAS.**
   - Performance: low travel time, safe route, accessibility, low number of wrong turns.
   - Environment: campus map, temporary blockages, weather, pedestrian traffic, user preferences.
   - Actuators: route suggestions, alerts for diversions, spoken/text instructions.
   - Sensors: GPS, map data, traffic updates, user destination, accessibility settings.
2. **Environment classification.** This is partially observable (GPS can be noisy), stochastic (crowds and closures change), sequential (wrong turns affect later steps), dynamic, and multi-agent.
3. **Suitable agent type.** A simple reflex agent is too weak because the best route depends on remembered context and changing conditions. A model-based or utility-based agent is more suitable because it must trade off time, safety, and accessibility.
4. **Likely failure mode.** If the reward or utility overemphasizes shortest distance and ignores safety, the system can recommend risky paths even when it appears "efficient".

9 Environment properties

We often classify environments along several dimensions:

- **Fully observable** vs **partially observable**: does the agent see the complete state?
- **Deterministic** vs **stochastic**: does an action have a predictable outcome?
- **Episodic** vs **sequential**: do current decisions affect future states?
- **Static** vs **dynamic**: does the world change while the agent deliberates?
- **Discrete** vs **continuous**: are states/actions/time steps countable?
- **Single-agent** vs **multi-agent**: are there other agents with possibly competing goals?

9.1 Examples for environment properties

- **Chess:** fully observable, deterministic, sequential, static, discrete, multi-agent.
- **Driving:** partially observable, stochastic, sequential, dynamic, continuous, multi-agent.
- **Vacuum world (simple):** can be partially observable (if sensor limited), often deterministic in toy models.

9.2 Why classification matters

If an environment is partially observable and stochastic, the agent may need:

- memory (internal state) and
- probabilistic reasoning / learning

instead of simple reflex rules.

10 Rationality and bounded rationality

10.1 Rationality

An agent is **rational** if it selects actions that maximize expected performance given the percept history and the agent's knowledge. Key clarifications:

- Rational $\neq$ omniscient: it cannot know the true state perfectly.
- Rational $\neq$ perfect: limited by sensors, compute, time, and data.

10.2 Bounded rationality

In real systems, decisions must be made under constraints. A bounded-rational agent aims for the best feasible action under limited resources.

11 Agent types (high level)

- **Simple reflex agents:** use condition-action rules based only on current percept.
- **Model-based reflex agents:** keep internal state to deal with partial observability.

- **Goal-based agents:** select actions based on goal achievement.
- **Utility-based agents:** maximize a utility function (trade-offs, preferences).
- **Learning agents:** improve their behavior from experience.

11.1 Simple reflex vs model-based reflex (expanded)

Simple reflex uses rules like:

IF condition THEN action

This is fast and simple but fails when the right action depends on information not in the current percept.

Model-based reflex maintains an internal state (a simple "memory" of the world) so it can act sensibly under partial observability.

11.2 Goal-based and utility-based agents (expanded)

- **Goal-based agents** select actions to reach a goal state. They need some representation of the goal and a way to predict action outcomes (a model).
- **Utility-based agents** handle trade-offs (time vs cost vs risk) by maximizing a utility function. This is practical when there are multiple competing objectives.

12 Learning agent architecture

The learning agent can be described using four components:

- **Performance element:** produces actions (current policy/behavior).
- **Critic:** evaluates how well the agent is doing (feedback signal).
- **Learning element:** updates the performance element using feedback.
- **Problem generator:** suggests exploratory actions to gain useful experience.

12.1 Exploration vs exploitation (important intuition)

- **Exploitation:** choose actions that are currently believed to be best.
- **Exploration:** try new actions to discover better strategies.

The problem generator encourages exploration so that the agent does not get stuck with a suboptimal behavior early.

13 Common pitfalls (read before exams)

- **Confusing performance measure with reward:** reward is a designed signal; performance measure is what we truly care about. Misalignment causes "reward hacking".
- **Calling an environment deterministic when it is not:** real-world systems (traffic, markets, users) are usually stochastic and dynamic.
- **Assuming full observability:** most practical agents must act with partial and noisy information.

- **Forgetting that rational is conditional:** rationality is defined *given* the available percepts, actions, and objectives.

14 Practice (with answers)

P1. PEAS design

Write a PEAS description for an **online exam proctoring assistant**. **Answer (example):** P = minimize cheating, minimize false accusations, low latency; E = exam session, candidates, network; A = flag events, record clips, alert human proctor; S = webcam, screen activity logs, keystroke patterns (if allowed), timing.

P2. Environment classification

Classify the environment of a **food delivery rider routing app** (observable? deterministic?). **Answer:** partially observable (limited traffic signals), stochastic (traffic changes), sequential, dynamic, continuous, multi-agent.

15 Exit questions (with answers)

Q1. What is PEAS? Create PEAS for a campus navigation app.

Answer: PEAS is Performance, Environment, Actuators, Sensors. Campus navigation app: P = minimize travel time, avoid unsafe routes, accuracy; E = campus map, traffic, user constraints; A = route suggestions, notifications; S = GPS, map data, user preferences, traffic updates.

Q2. Explain partially observable vs fully observable environments with examples.

Answer: Fully observable: agent can see complete state (e.g., chess board). Partially observable: agent sees only part/noisy view (e.g., driving in fog; medical diagnosis).

Q3. Why is rational not the same as omniscient?

Answer: Rationality is about choosing the best action given available information and constraints; omniscience means knowing the true state/outcomes, which is unrealistic.

Q4. Compare any three agent types.

Answer: Simple reflex uses immediate rules; model-based adds memory of state; goal-based plans actions to reach a goal; utility-based optimizes trade-offs using utilities.

16 Summary

- Agents perceive and act; PEAS specifies tasks clearly.
- Environment classification helps choose suitable AI methods.
- Learning agents improve behavior through feedback and exploration.

Sources: Rich and Knight, *Artificial Intelligence*, McGraw Hill, 2017.;Russell and Norvig, *Artificial Intelligence: A Modern Approach* (4e), Ch. 1–2 (Introduction, Intelligent Agents).;Poole and Mackworth, *Artificial Intelligence: Foundations of Computational Agents* (2e), selected sections.